1

学习的方法

第一章

你自己就可以是一家公司

自身行为产生的后果归根结底都是思考的产物，因此，你要想规划好自己的行为结果就要控制自己的思想，这是根本所在。

——查尔斯·哈奈尔

如果你对这个世界的理解是对的，那你的口袋里应该已经有了你想要的东西，否则就应该停下来，认真回想一下过去的生活。是不是哪里出了问题？要不要沿着之前的轨迹继续生活下去？

很多人在离开学校以后就停止学习了，之后只是在不停地重复之前的行为。

爱因斯坦说：“疯狂就是重复做一件事，却又期待不一样的结果。”

改变没那么难，你需要掌握的仅仅是几个关键概念和最基础的算法而已。

函数（方法）

首先我想介绍一个编程领域的概念给你，叫作函数。

别担心，我们不用写程序。引入一个新概念是为了让整本书的内容更容易而不是更难理解，方便你学以致用。

最终我决定使用“函数”，是因为这种更贴近机器的语言风格可以让我们把更多的注意力放在最关键的内容上，而不是遣词造句上，让我们的注意力更集中。虽然一开始你可能会经历那么一点“学习曲线”，或者有不适应的感觉。

如果这让你感受到压力，记住这种感觉。在学习过程中，有那么一点点

不舒服是好事，那意味着你走出了之前的舒适区。走出原有的舒适区是进步的开始。从单纯追求舒适的角度来说，躺在床上什么也不干无疑要舒适得多。

进步的核心，不是日复一日地重复昨天的事，而是不断尝试新的挑战。

什么是函数？函数是一个最基本的可运行单位。在某种意思上，你也可以把它理解为“方法”。

在我看来，所有的知识都应该是可运行的，也就是可以被应用出来的。否则，我们学习它是为了什么呢？

我们可以把那些用来解决具体问题的“算法”封装到一个函数（方法）里。

比如“加法”——add()，可以将两个具体的数字相加。

在实际的应用场景中，比如便利店收费，每扫描一件商品，就会自动将它的价格与之前的金额相加。扫描第一件商品时，得到它的价格为 1；扫描第二件商品时，得到它的价格为 2；这个时候收银台会自动根据扫描的商品计算出我们需要支付的消费金额。

其实这是因为收银台在后台默默调用了一个加法运算：

```
add(1,2) = 3;
```

add 代表方法。

add( ) 代表方法的一次运行。对应便利店的两次扫码，第一件商品的价格是 1，第二件商品的价格是 2；显示需要支付的消费金额为 add(1,2) = 3。如果第一件商品的价格是 3，第二件商品的价格是 4，则会显示需要支付的消费金额是 add(3,4) = 7。

add(a,b)=a+b，是后台设定好的具体算法。对外不可见，也就是说用户并

不知道是怎么计算的。当他扫码消费时，add 方法会默默调用 a+b 这个算法，由收银台显示算法运行的结果。

我们可以把函数（方法）当作一个处理器，输入什么，经过处理，得到什么。

在 add( ) 这个例子中，输入的是“1,2”，经过 add 函数运算，得到运行结果 3。如果输入“3,4”，经过 add(3,4) 方法运算，输出结果为 7。

生活中不只有加法，还有其他各式各样的用来处理不同问题的方法。

比如，一个“很乱的房间”，经过“打扫卫生 ( )”，得到的应该是一个“干净整洁的房间”；一个“糟糕的企业现状”，经过“咨询诊断 ( )”，得到了一套具体可实操的“解决方案”；“100 万的资本”，经过一年的“投资 ( )”，最终变成了“200 万”，当然也可能是“50 万”，这取决于你的投资水平。

这些方法对应的是你的能力。能力越高，获得的收益自然就越大。

我们应该多学习一些可以帮助我们改变生活，让我们的生活变得更美好的方法。

我在这本书里为你提供了一些，如“产品设计 ( )”“营销 ( )”“投资 ( )”。

每个大的算法里又包含了很多具体的小的实操算法，如“选股 ( )”“估值 ( )”“选择时机 ( )”等，就像这样：

投资( ) = 识别公司( ) + 估值( ) + 选择时机( ) + ……

掌握 IPO，就可以开公司

这是一个输入、输出的过程。

输入“100 万”，得到“200 万”。

输入“一本书”，得到一个可以换取“100 万”的“解决方案”。

你自己就可以是一家公司，有可能还不止一家。

比如我，其实是一家“小型电视台”，每天都有一些固定的知识类节目，每周二会有一场直播。收视率最高时，我的节目大概有超过 2000 万的播放量。

我还开了一家“小型书店”，里边放了一些我自己读过的、喜欢的书，几十本的样子。因为我对内容有过滤，价格公道，经常有一些“老伙计”光顾。

我还是一个“很小的杂志社”，偶尔写几篇专栏，如果写得好，会收到不同数目的稿费。

我还开设了一家“健脑俱乐部”，帮助人们提升认知和解决问题的能力，收取年费。

我还开设了一家“小型咨询公司”，为一些企业提供咨询服务。

并且我还是一名“投资人”，是数家小型公司和上市公司的股东。

你自己就可以是一家公司，也就是 IPO 模式：

Input( )→Process( )→Output( )

不过是一些方法的执行罢了。不需要托人送礼、找关系，不需要看别人的脸色，通通不需要，你自己一个人就可以搞定。

我还有一些其他的生意，但那都不过是我掌握的一些方法（为了方便大家理解，之后的章节统一把“函数”称为“方法”）而已。

这本书就是给你介绍这些方法的（你可以直接翻到后面的章节简单了解一下）：“产品设计()”“营销()”“投资()”……下载它们，安装到你的“操作系统”上，需要的时候完成一次方法调用就可以了。

提取自己的成功方法

你也可以尝试着自己提取“方法”。首先是提取那些以往曾经帮助过自己获得成功的方法。

如果你曾经做出过一盘非常好吃的蛋炒饭，只要你愿意思考、复盘，以后你做的蛋炒饭都会很好吃；如果你曾经写出过一篇非常棒的文章，只要你愿意思考、复盘，以后你写的文章都会维持在一定的水准。

有些事，你成功过，只要你愿意思考、复盘，大概率是可以复制的。相似条件越多，越容易复制。

有事没事看看自己，想想看：取得过哪些成功，这些成功可以复制吗？将获得此次成功的条件和算法提取出来，反复做，让它成为自己的习惯。

把整个过程记录下来，总结出一套基础方法。想办法把方法中所有不相关的因素去掉，提取“变量”（可能影响结果的不稳定因素），一一进行试

错，根据结果反馈复盘调整。

```
识别过程 ( );
识别成功的条件 ( );
场景再现 ( );
如果可再现，提取方法 ( );
```

在这个方法的基础上，持续试错、调整、重构，持续改进。

提取痛苦清单

你肯定不只做过成功的事，一定也做过失败的事。找到它，思考一下，有没有什么可以避免的方法，把它整理进“痛苦清单”。

约翰尼·卡森在哈佛大学的毕业演讲中，开出了“如何保证自己过上痛苦生活”的痛苦清单：

```
为了改变心情或者感觉而使用化学物质；
妒忌；
怨恨。
```

在此之后，查理·芒格在哈佛毕业典礼的演讲中扩展了这个清单：

> 要反复无常，不要虔诚地做你正在做的事；
>
> 不要从其他人成功或失败的经验中广泛地吸取教训，不管他们是古人还是今人；
>
> 遭遇失败时，请意志消沉，从此一蹶不振吧；
>
> 请忽略朴素的智慧。

照这个清单做，你就可以获得痛苦。

（在第二十二章《投资者的错误行为》中，我列举了更详细的错误行为以及避免这些错误的方法。你可以直接翻到这一部分，思考如何列出属于你自己的“痛苦清单”。）

提取他人的成功方法

注意查理·芒格给出的痛苦清单的第 2 条：

“不要从其他人成功或失败的经验中广泛地吸取教训，不管他们是古人还是今人。”

你不是第一个来到这个星球的人。很多事情，早就有人做过了，没必要

自己一个一个地从头摸索。找到那些比自己优秀的人，向他们学习，尤其是那些看起来条件和你差不多的人。

“先像专家一样学会规则，然后才能像艺术家一样打破它们。”

乔丹说：“如果我没有看过 J 博士在全盛时期的惊人演出，我就不可能拥有像现在一样的视野。”

科比则更加直接：“我场上所有的动作，都是从观看偶像球星的录影带中学来的。”

巴菲特经常阅读霍华德·马克斯的投资备忘录，而霍华德·马克斯每年都会仔细阅读巴菲特致股东的信，并在自己的书中和投资备忘录中多次引用巴菲特的名言。

闲暇的时候，比如坐地铁或者发呆，我喜欢看那些曾经做出过成绩的人的朋友圈。在我看来，一个能做成事的人一定在某些地方比我强。为什么他能做成，我做不成？

我喜欢向所有比我优秀的人学习。

早些年我喜欢看王兴的“饭否”，看多了对他的思想就有了一个大概的了解。我用从他那里学来的角度理解移动互联网，总是比身边的人快半步。

产品方面，我喜欢看张小龙，俞军在产品方法方面的演讲对我的帮助也很大。

个人成长方面，我反而喜欢看张一鸣，尤其是他 30 岁左右时写的微博。

我最早的演讲能力是从罗永浩那儿学的。十几年前，手机还没有这么普及，很多人用的还是 MP3，MP3 里装的都是“老罗语录”。我找到它们，下载下来，拿个本子一条一条过。他的演讲的结构是怎样的？他是怎么开场的？他是怎么从一个主题切换到另外一个主题的？为什么在这个地方听众会笑？没有观众，自己一个人在屋子里对着镜子演练，一条一条地临摹。

注意，我说的是临摹。我发现很多人无法分辨临摹和抄袭的区别。抄袭

往往是为了获得结果带来的好处，而临摹则是为了学习获得成功的方法。

接着我用同样的方式临摹乔布斯。

就是这样，先找一个你能够得着的人学习，然后换下一个。

我找到了市面上所有拆解乔布斯演讲的书籍，对照着乔布斯、罗永浩、TED 的演讲，抽取共同特性，将其转化为一个演讲方法。其“基础算法”如下：

> 定义问题，“为什么会有这场演讲 ( )”；
> 回答最重要的问题 ( )；
> 创作动人故事 ( )；
> 请“大坏蛋”出场 ( )；
> 给人们一个看世界的新视角 ( )；
> 使用超酷的词汇 ( )；
> 简化一切 ( )；

每当你发现一个在某些方面比自己更厉害的人时，就应向他们学习。看看他们在哪些方面比自己优秀，应该向人家学习什么。

像最早的皮革厂老板一样，看谁家的包设计得好，买回来，一点一点拆开，1∶1 复刻。不是山寨，是临摹。

所谓“三人行，必有我师”，谁比你厉害，就“临摹”他！

于是，你多了一位罗永浩这样的演讲老师，多了一位李笑来这样的学习老师，多了一位李欣频这样的文案老师，多了一位张一鸣这样的创业老师，多了一位李录这样的投资老师……

我有数百位这样的老师。

于是，你的演讲能力提升了，你的学习能力提升了，你的文案能力提升

了，你的产品能力提升了，你的投资能力提升了（在第三部分《投资的方法》中，我会为你演示“如何临摹他人投资框架”的完整过程）……

有一天在地铁上，我看到张一鸣的微博。他说：“今天听王兴演讲的比喻，人生，和谁一起在路上，看什么风景。我最近也在想，以后要让小孩多看看传记，包括电视剧《阿信》那样的也可以，看看别人的风景和旅程，更容易想清楚自己的选择。”

你看，张一鸣也是这么干的。

一次，有人问张一鸣最喜欢读什么书。张一鸣说，一种是教科书，一种是传记。

几年后，有人问我最喜欢读什么书。我说，一种是教科书，一种是传记。

我是真的这么认为。

教科书代表了一个领域最基础的知识和关键概念。

读传记可以从他人的生命中抽取他成功和失败的模式，看看那些做出过成绩的人在面对问题的时候，是如何思考和行动的。

如果有人因此取得过成功，并且大量的案例证明了这件事，你需要做的就是找到那个最简单朴素的模式，看看可以给你带来什么帮助。

重构

向那些比你优秀的人学习，提取他们成功的方法。

一开始不一定是正确的，这需要一个不断尝试、调整、验证的过程。

做不到也没关系。先做到 60 分，运行起来，然后再想办法做到 90 分、100 分、120 分。

编程领域有一个很重要的概念，叫重构，又被称为“童子军训练守则”：“离开露营地的时候，永远让它比你发现之前干净。”

每检入一次代码，就要让它比上次检出时变得更为简洁。每次读代码，都别忘了进行点滴的改善。

“重构 ()”是一个极重要的方法，其算法如下：

```
复盘 ( 当前方法 );
发现问题 ( 当前方法 );
变得更好 ( 当前方法 );
```

不要等它完美了再去运行。一边运行，一边让它完美。

也许最终你无法超越乔布斯的演讲，但那又怎么样呢？你的演讲能力实实在在地提升了，足以吊打市面上半数以上教演讲的老师。

我用同样的方法学习了视觉、戏剧、引导、产品设计、营销、投资、新媒体……

从那些比你优秀的人的身上学习——对！仅仅是学习——然后照着做，就能收获很好的知识。

“如果乌龟能够吸取它那些最棒前辈的已经被实践所证明的洞见，有时候它也能跑赢那些追求独创性的兔子，或者跑赢宁愿跻身忽略前人最优秀的工作的蠢货之列的兔子。”（查理·芒格）

你需要做的不过是把那些已经被证明了的应用方法，安装到你的操作系统上，需要时完成一次方法调用就可以了。唯一的问题是，很多人的“操作系统”在离开学校以后，就停止更新了，之后只是在不停地重复之前的行为而已。

绝大部分兔子在跑到一半时，就躺在大树下睡着了——俗称“躺平”，以至于操作系统的版本过低，很多应用安装不上。

再好的软件也无法安装到低版本的操作系统上，虽然那确实也叫操作系统。

自学

你也可以把这本书理解为“一个普通人的操作系统搭建与应用软件安装说明书”。

打开这本说明书的第 1 页，你要安装的第一个也是最重要的算法应该叫“自学 ( )”。

它是所有算法的基础，我称之为元算法。几乎所有其他类型的算法都需要通过该算法获得，边学边用，边用边学。用一次“重构 ( )”，下次使用时

效率会更高。如此迭代下去，人自然会变得优秀。

你完全可以把它理解为一个安装程序：

```
安装 ( 产品设计方法 );
安装 ( 营销方法 );
安装 ( 投资方法 );
```

有了它之后，你就可以在自己的操作系统中安装好多应用。

比如，你要投资，正确的过程应该是：

```
自学 ( 投资 ); // 于是你学会了投资
投资 (100 万 ); // 现在你可以调用它了
```

可很多人是怎么做的呢？

```
投资 (100 万 );
```

他们发现自己并没有投资这个函数。怎么办呢？于是随便找了个函数（有可能算法都是错的）安装上，欺骗自己：“我已经掌握了。”

人是很会欺骗自己的。

也是很擅长欺骗的。

比如上学这件事，很多人一直没有搞清楚上学和学习的区别。

1. 小明一直在上学，但他并没有学习。

2. 小明一直在学习，但他并没有上学。

上学是学校帮你升级操作系统并安装应用。最后有没有升级成功、是不

是安装上了，并不取决于你上没上学，只能说成品率可能会高一些。

自学是你自己升级操作系统并安装应用。

想想就激动。学校是批量操作啊！它安装的是大众版本的操作系统和应用。

“学校的存在就是为了尽可能地使人们具有统一性。学校就像一种模具。孩子们在很小的时候就被送到这里，被教育成标准的模样，从头到尾都像是被盖上了官方的橡皮图章。”（亨利·路易斯·门肯）

要想成为你自己，你需要学会自学，也必须自学。

那些真正优秀的人很早就明白了这个道理。所以他们哪怕是在上学，也是以自学为主的。

看见自己

你有没有发现，那些在生活中很“被动”（意愿）的人，做事也总是很被动的？

他们希望别人为他们升级操作系统（“我应该考研还是上 MBA？”），希望别人为他们安装应用函数（“帮我看看那个培训班是不是‘割韭菜’的？”），希望别人为他们做选择（“帮我推荐几只股票呗？”）。

……

他们希望别人为他们做一切他们不想做的事，而他们只需要收获最后的成果就好（“这不是我工作职责范围以内的事。”“凭什么他们有，我没有？”）。

于是，他们什么都不需要做，只需要做一件事——做梦就好了。当然还有一件事，那就是发牢骚。

而主动的人会主动寻找自己身上的问题，然后加以改进。

让我们回到这篇文章的开头。还记得吗？如果你感觉到了压力，记住这种感觉，它非常重要。这是我要和你分享的一个极为重要的算法，我叫它“看见自己”。

“编程？我不会啊。”

“这是什么？能不能简单点？搞那么复杂。”

“算了吧，我上学的时候理科成绩就不好。”

……

这些都说明你的身体里有个缺陷，它是造成你止步不前的一个重要原因。每当你准备往前迈一步的时候，它就会拉扯你。就像那些生活中看不得你好的人一样。

它很坏！死死盯着它。审视这个缺陷，你看见了“自己”。

当你“看见自己”的时候，你就看到了那个缺陷，就有了进一步改进的空间。

这是一个极为重要的人生算法：“看见自己”。当你能“看见自己”时，你就有机会在自己陷入困境之前，按下“暂停”键。

还记得是什么让你的生活陷入困境的吗？

1. 为了改变心情或者感觉而使用化学物质。

2. 妒忌。

3．怨恨。

4．反复无常，不要虔诚地做你正在做的事。

5．不从其他人成功或失败的经验中广泛地吸取教训。

6．遭遇失败时，意志消沉，从此一蹶不振吧。

7．忽略朴素的智慧。

8．拒绝改变。

……

这些都是让你的生活陷入困境的方法。

如果你认真观察，就会发现这个现象：让你的生活陷入困境的不是你没做某事，而是你做了不该做的事，是那些缺陷让你的生活陷入了困境。你需要及时发现它们，否则你一定会为此付出代价。

“如果你的后卫线有漏洞，那我就会一直点名打你的后卫。”

“如果你的右腿有伤，那我就会一直攻击你的右腿。”

“如果你好色，那我就色诱你。”

“如果你贪图小便宜，那我就会送一些小便宜给你。”

……

当你看不清自己，总想破格获取一些自己能力之外的东西时，你一定会陷入困境。看看那些背负债务的人，看看那些在金融市场中投资失败的人，你就能明白这一点。

如果你有一个缺陷，那这个缺陷一定会最大限度地反复运行，直到你发现并修正它为止。

我在本书里帮助你罗列了一些在你身上可能存在的缺陷，以及由此可能造成的致命伤害。当你翻到后文《投资者的错误行为》这一章节的时候就会看到它们。

只需要把这些缺陷罗列成一个清单，在行动之前，比如在你准备买入一

家公司之前，对照着清单检查一遍，就可以减少你 90% 以上的损失。

就是这么神奇。

“看见自己”很重要，它是你发现自己的过程。

“你不知道你，所以你是你。如果你知道了你，你就不是你了。”

有人给它起过一些别的名字，比如“内观”“反省”“元认知”什么的。我觉得这些词都太大了，简单一点，“看见自己”。

看到了以后，不用总想着马上改变它。

```
暂停 ( );
复盘 ( );
反思 ( );
```

看见，就很好。

基础算法

“看见自己 ( )”“自学 ( )”“重构 ( )”是三个最基本的算法。

我甚至认为你在阅读后面的每一章之前，都需要调用一下“看见自己 ( )”这个方法；在阅读完成之后，调用一下“重构 ( )”这个方法。

```
看见自己 ( );
自学 (x) ; // 学习最少且重要的知识
重构 (x) ;
```

拥有了这三个算法，不代表你就拥有了一个高版本的操作系统，但意味着你拥有了持续升级到更高版本操作系统的可能。

“看见自己 ()”可以帮助你发现系统中的缺陷。

“自学 ()”可以帮助你升级和安装更好的操作系统和应用。

“重构 ( )”可以让你在原有的基础上持续迭代。你不需要一开始就有多好，但可以变得越来越好。

于是，你每天只需要往前挪一小步就可以了：

$$(1+0.01)^{365} \approx 37.78 > 1$$

是真的只需要挪一小步。你很快就能发现这一点。